Alternative Assessment Tool (AAT) Report on

“Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL”

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Shreya Kabadagi **1BM24EC197**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

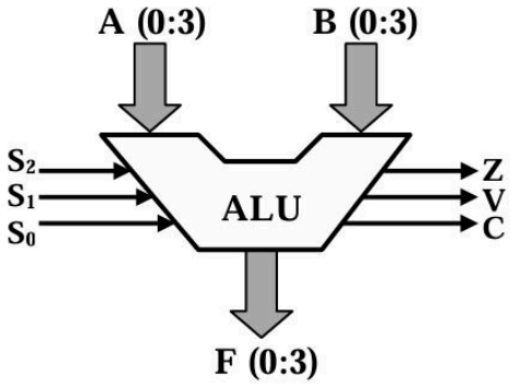
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

1) Arithmetic Logic Unit (ALU)

DESIGN:



Z, **C** and **V** are status flags
Z = 1 if **F** = 0
V = Overflow
C = Carry or Borrow

S_2	S_1	S_0	Function (F)
0	0	0	A + B
0	0	1	A – B
0	1	0	A – 1
0	1	1	A + 1
1	0	0	A **AND** B
1	0	1	A **OR** B
1	1	0	**NOT** A
1	1	1	A **XOR** B

VERILOG CODE:

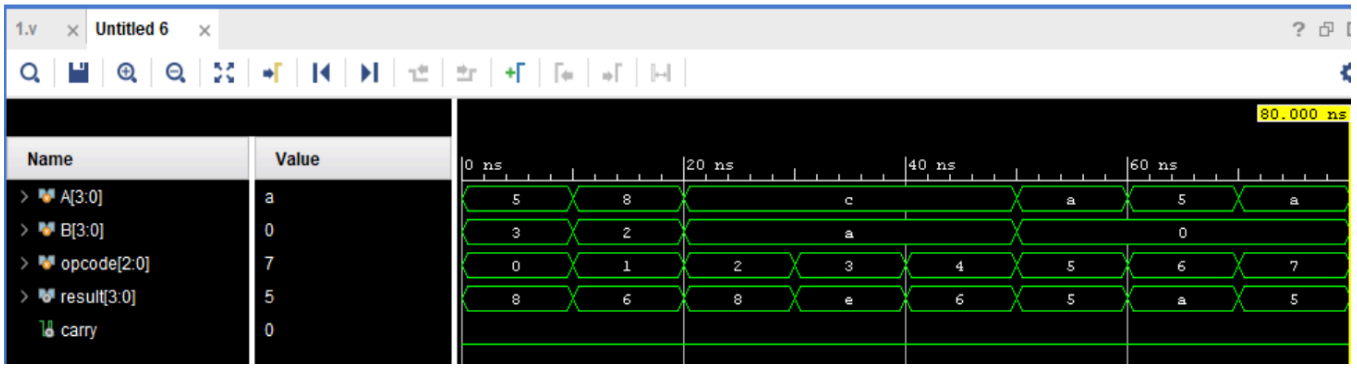
```
 module ALU (
    input  [3:0] A,
    input  [3:0] B,
    input  [2:0] opcode,
    output reg [3:0] result,
    output reg carry
);
always @(*) begin
        result = 4'b0000;
        carry  = 1'b0;
        case (opcode)
            3'b000: {carry, result} = A + B;    // Addition
            3'b001: result = A - B;             // Subtraction
            3'b010: result = A & B;             // AND
            3'b011: result = A | B;             // OR
            3'b100: result = A ^ B;             // XOR
            3'b101: result = ~A;                // NOT
            3'b110: result = A << 1;            // Left shift
            3'b111: result = A >> 1;            // Right shift
```

```
      default: begin
            result = 4'b0000;
            carry  = 1'b0;
           end
     endcase
 end
endmodule
```

TESTBENCH:

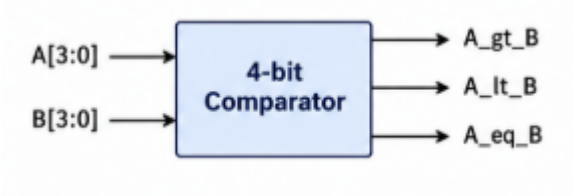
```
module ALU_TB;
reg [3:0] A;
reg [3:0]B;
reg [2:0]opcode;
wire [3:0] result;
    wire carry;
    ALU uut (A,B,opcode,result,carry);
       initial begin
       $monitor("Time=%0t A=%b B=%b Opcode=%b Result=%b Carry=%b",
                $time, A, B, opcode, result, carry);
A = 4'b0101; B = 4'b0011; opcode = 3'b000;
#10 A = 4'b1000; B = 4'b0010; opcode = 3'b001;
#10 A = 4'b1100; B = 4'b1010; opcode = 3'b010;
#10  A = 4'b1100; B = 4'b1010; opcode = 3'b011;
#10  A = 4'b1100; B = 4'b1010; opcode = 3'b100;
#10  A = 4'b1010; B = 4'b0000; opcode = 3'b101;
#10  A = 4'b0101; B = 4'b0000; opcode = 3'b110;
#10  A = 4'b1010; B = 4'b0000; opcode = 3'b111;
#10   $finish;
 end
 endmodule
```

OUTPUT:



2) Comparator

DESIGN:



A (4-bit)	B (4-bit)	A_gt_B	A_lt_B	A_eq_B
A > B	–	1	0	0
A < B	–	0	1	0
A = B	–	0	0	1

VERILOG CODE:

```
module comparator (
   input [3:0] A,
   input [3:0] B,
   output reg A_gt_B,
   output reg A_lt_B,
   output reg A_eq_B );
always @(*) begin
A_gt_B = 0;
A_lt_B = 0;
A_eq_B = 0;
if (A > B)
     A_gt_B = 1;
else if (A < B)
     A_lt_B = 1;
else
    A_eq_B = 1;
end
endmodule
```

TESTBENCH:

```
module comparator_TB;
reg [3:0] A, B;
wire A_gt_B,A_lt_B,A_eq_B;
comparator uut (A,B,A_gt_B,A_lt_B,A_eq_B);
initial begin
$monitor("Time=%0t A=%d B=%d A_gt_B=%b A_lt_B=%b A_eq_B=%b",
              $time, A, B, A_gt_B, A_lt_B, A_eq_B);
A = 4'd10; B = 4'd5;
#10   A = 4'd3; B = 4'd8;
#10  A = 4'd7; B = 4'd7;
```

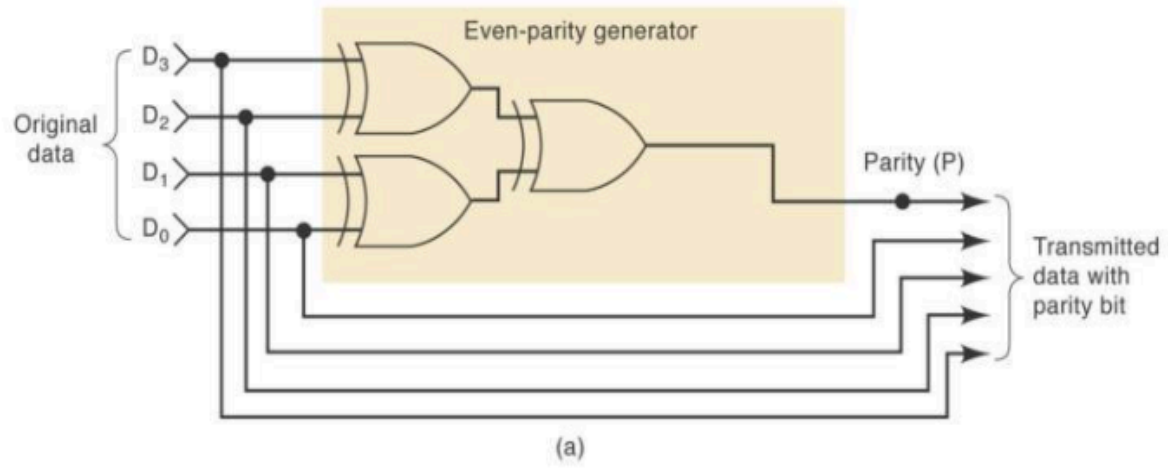
```
#10   A = 4'd15; B = 4'd2;
#10    A = 4'd1; B = 4'd12;
#10 $finish;
end
endmodule
```

OUTPUT:

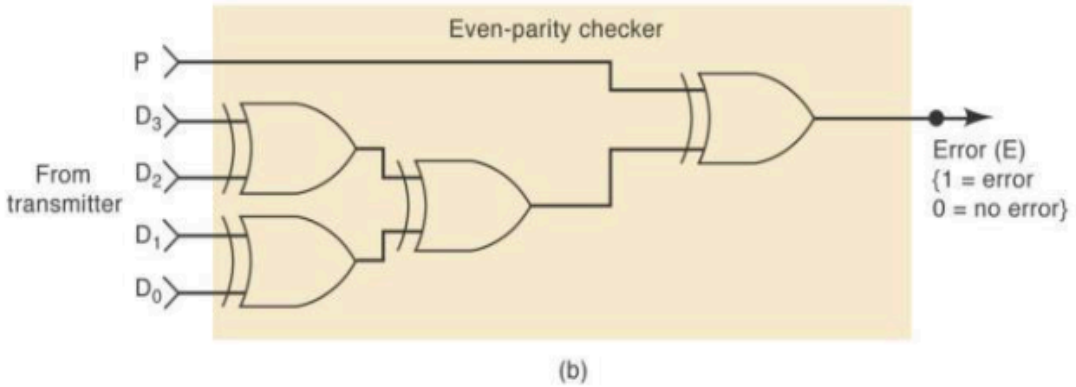


3) 4-Bit Parity Generator and Checker

DESIGN:



(a)



(b)

VERILOG CODE:

```
module parity_generator_checker (
        input [3:0] data,
        input parity_in,
        input parity_type,
        output even_parity,
        output odd_parity,
        output error );
      // Parity Generator
        assign even_parity = ^data;
        assign odd_parity  = ~(^data);
        // Parity Checker
        // parity_type = 0: Even parity
        // parity_type = 1: Odd parity

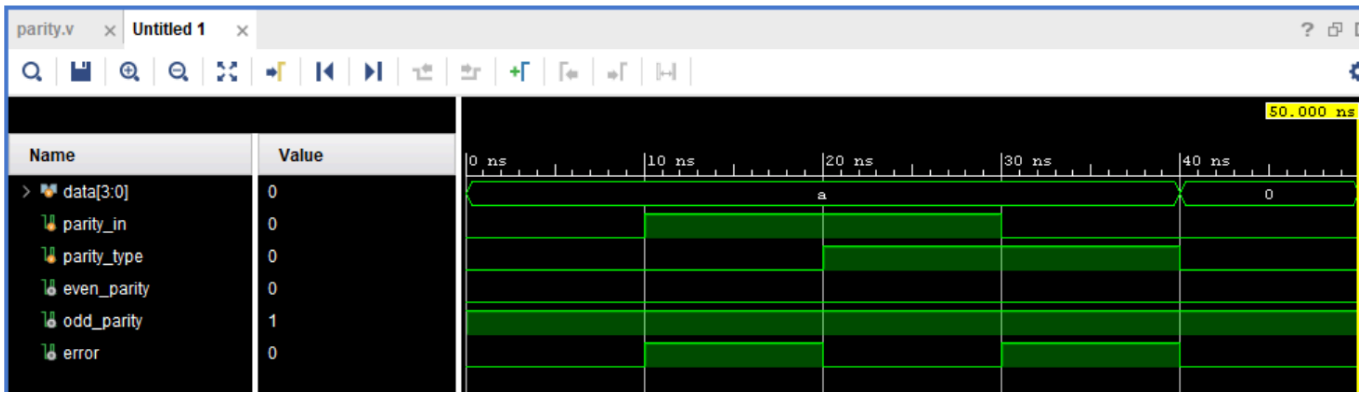
        assign error = (parity_type == 1'b0)
                ? ^{data, parity_in}
                : ~^{data, parity_in};
endmodule
```

TESTBENCH:

```
      module parity_TB;
      reg [3:0] data;
      reg parity_in;
      reg parity_type;
      wire even_parity;
      wire odd_parity;
      wire error;
      Parity_generator_checker
      uut(data,parity_in,parity_type,even_parity,odd_parity,error);
      initial begin
            data = 4'b1010;
            parity_type = 1'b0;
            parity_in = 1'b0;
            #10; data = 4'b1010;
            parity_type = 1'b0;
            parity_in = 1'b1;
            #10; data = 4'b1010;
            parity_type = 1'b1;
```

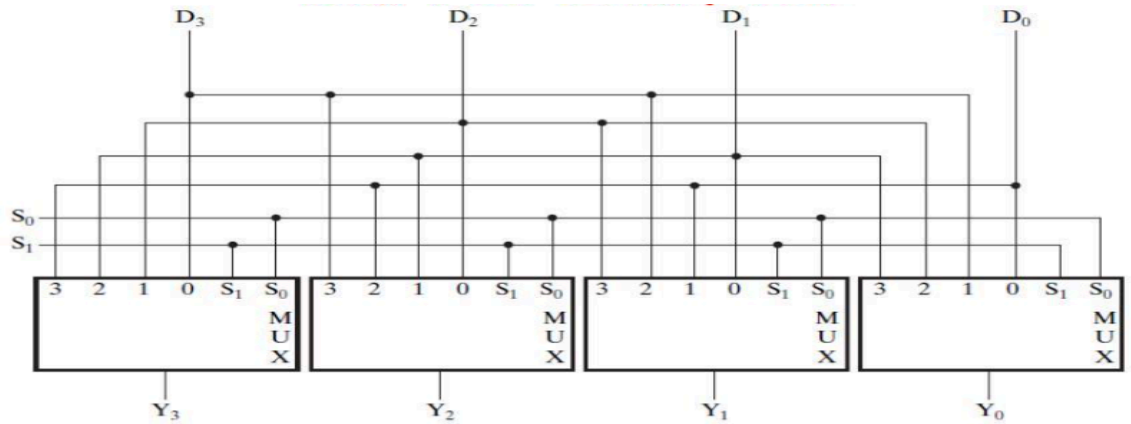
```
        parity_in = 1'b1;
        #10; data = 4'b1010;
        parity_type = 1'b1;
        parity_in = 1'b0;
        #10; data = 4'b0000;
        parity_type = 1'b0;
        parity_in = 1'b0;
        #10; $finish;
    end
    endmodule
```

OUTPUT:



4) 4-Bit Barrel Shifter

DESIGN:



Function Table for 4-Bit Barrel Shifter

Select		Output				
S_1	S_0	Y_3	Y_2	Y_1	Y_0	Operation
0	0	D_3	D_2	D_1	D_0	No rotation
0	1	D_2	D_1	D_0	D_3	Rotate one position
1	0	D_1	D_0	D_3	D_2	Rotate two positions
1	1	D_0	D_3	D_2	D_1	Rotate three positions

VERILOG CODE:

```
module barrel_shifter (
   input [3:0] data_in,
   input [1:0] shift,
   input direction,
   output reg [3:0] data_out);
always @(*) begin
    case (direction)
     1'b0: begin
             case (shift)
                2'b00: data_out = data_in;
                2'b01: data_out = data_in << 1;
                2'b10: data_out = data_in << 2;
                2'b11: data_out = data_in << 3;
             endcase
          end
     1'b1: begin
             case (shift)
                2'b00: data_out = data_in;
                2'b01: data_out = data_in >> 1;
                2'b10: data_out = data_in >> 2;
                2'b11: data_out = data_in >> 3;
       endcase
          end
        endcase
      end
endmodule
```

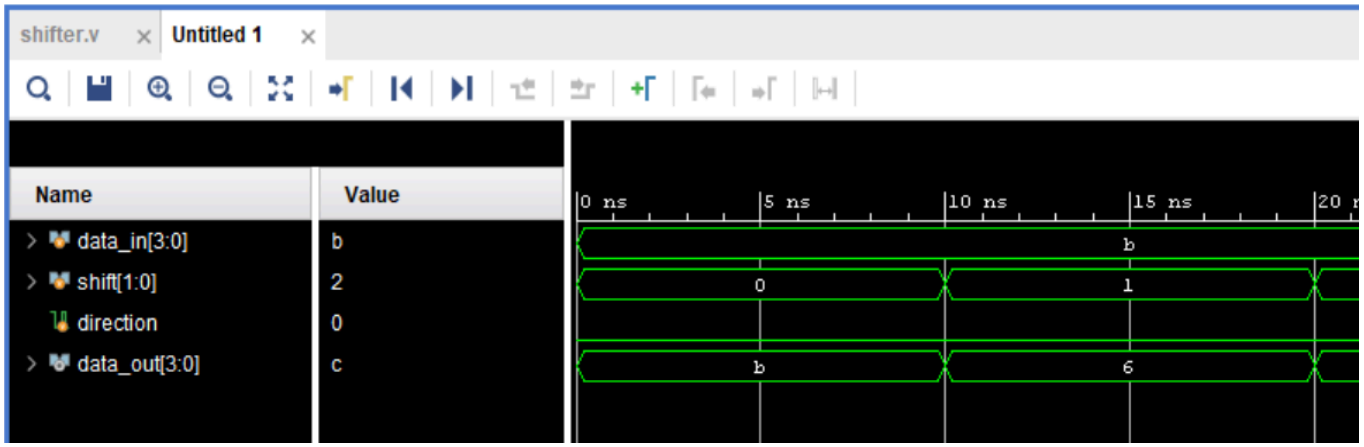
TESTBENCH:

```
module barrel_shifter_TB;
reg [3:0] data_in;
reg [1:0] shift;
reg direction;
wire [3:0] data_out;
barrel_shifter uut (data_in,shift,direction,data_out);
   initial begin

monitor("Time = %0tData = %bDirection = %bShift = %bOutput = %b",
time, data_in, direction, shift, data_out);
```

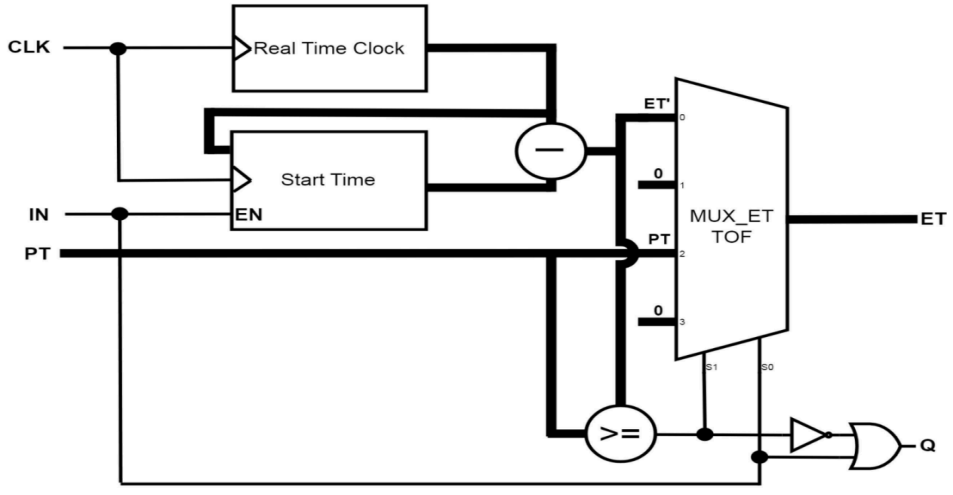
```
        // Test 1: No shift
        data_in = 4'b1011;
        direction = 0;
        shift = 2'b00;
        // Test 2: Left shift by 1
 #10       data_in = 4'b1011;
        direction = 0;
        shift = 2'b01;
        // Test 3: Left shift by 2
  #10      data_in = 4'b1011;
        direction = 0;
        shift = 2'b10;
   #10   $finish;
     end
endmodule
```

OUTPUT:



5)Timer Module

DESIGN:



VERILOG CODE:

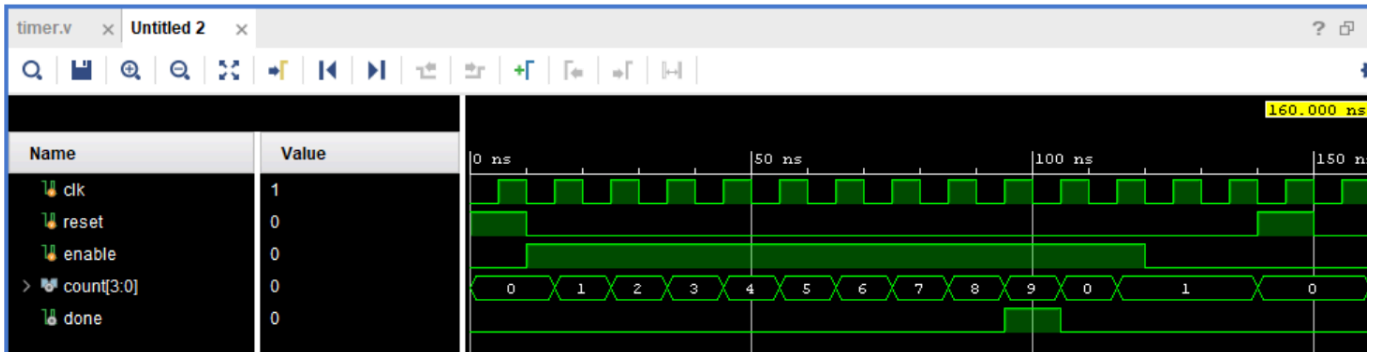
```
 module timer (
    input clk,
    input reset,
    input enable,
    output reg [3:0] count,
    output done
);
    // Timer terminal count
    assign done = (count == 4'd9);
     always @(posedge clk or posedge reset) begin
        if (reset)
            count <= 4'd0;
        else if (enable) begin
            if (count == 4'd9)
                count <= 4'd0;
            else
                count <= count + 1'b1;
        end
    end
endmodule
```

TESTBENCH:

```
 module timer_TB;
    reg clk;
    reg reset;
    reg enable;
    wire [3:0] count;
    wire done;
    timer uut (clk,reset,enable,count,done);
    always #5 clk = ~clk;
      initial begin
        clk = 0;
        reset = 1;
        enable = 0;
        $monitor(
            "Time=%0t Reset=%b Enable=%b Count=%d Done=%b",
            $time, reset, enable, count, done);
        #10;
```

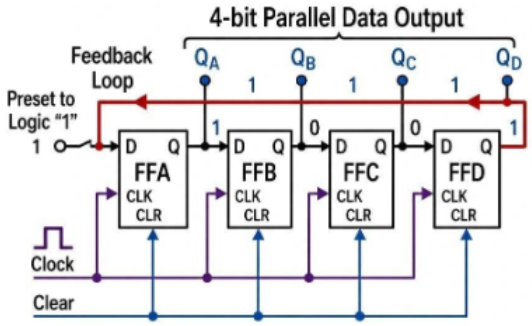
```
        reset = 0;
         enable = 1;                          // Enable the timer
      #110;      // Allow the timer to count
 // Disable the timer
        enable = 0;
 #20;
// Reset the timer again
        reset = 1;
        #10;
reset = 0;
#10;
        $finish;
    end
endmodule
```

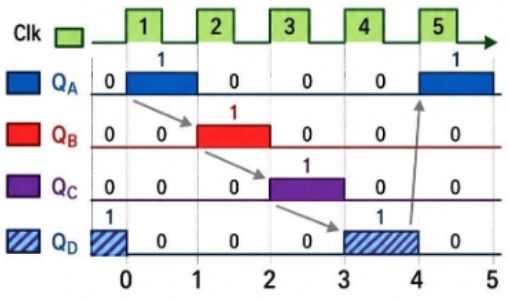
OUTPUT:



6) RING COUNTER

DESIGN:





VERILOG CODE:

```
module ring_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
```

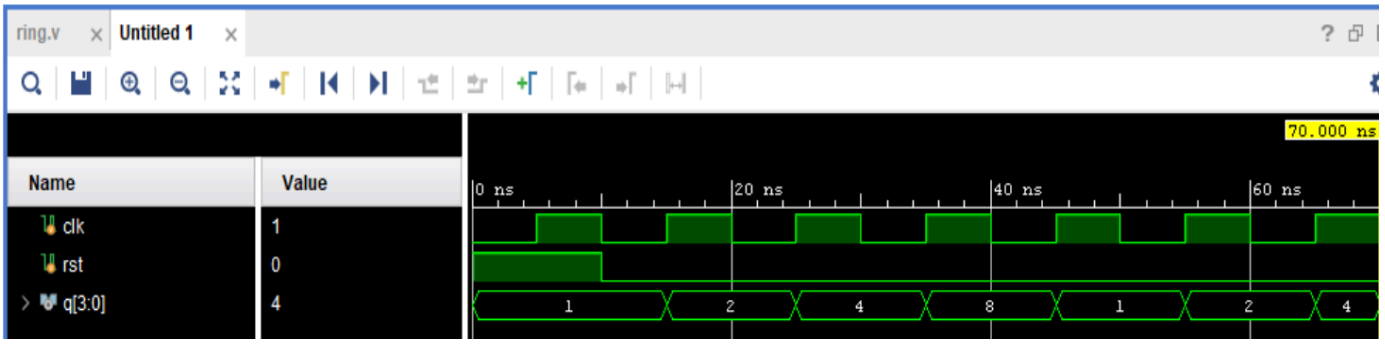
```
  always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0001;
        else
            q <= {q[2:0], q[3]};
    end
endmodule
```

TESTBENCH:

```
module ring_counter_tb;
    reg clk, rst;
    wire [3:0] q;
ring_counter uut (.clk(clk), .rst(rst), .q(q));

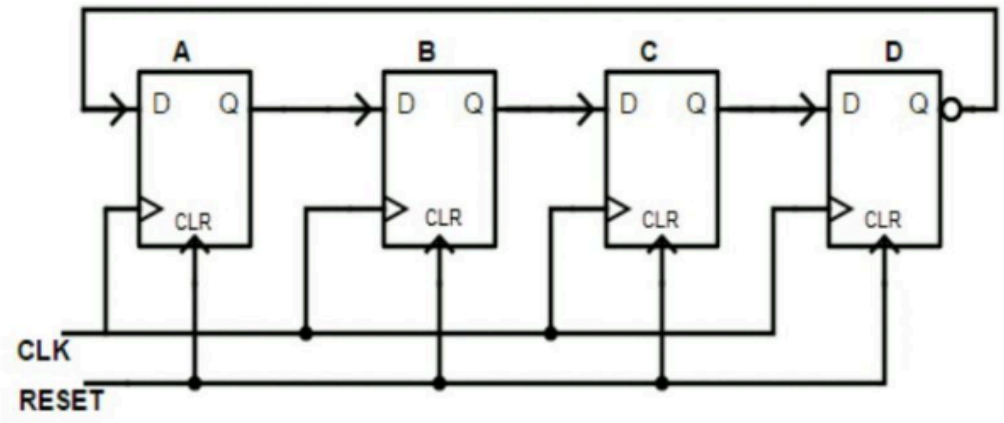
    always #5 clk = ~clk;
    initial begin
        clk = 0; rst = 1; #10;
        rst = 0; #60;
        $finish;
    end
endmodule
```

OUTPUT:



7) Johnson counter

DESIGN:



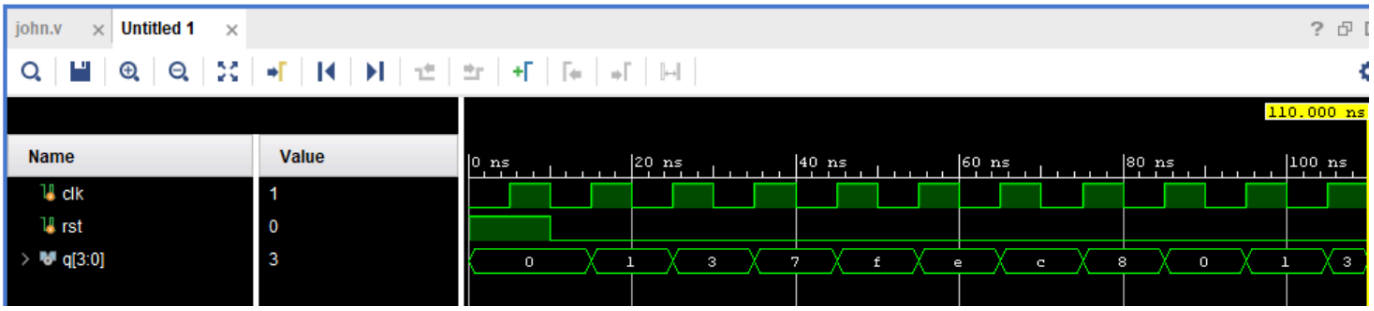
VERILOG CODE:

```
module johnson_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {q[2:0], ~q[3]};
    end
endmodule
```

TESTBENCH:

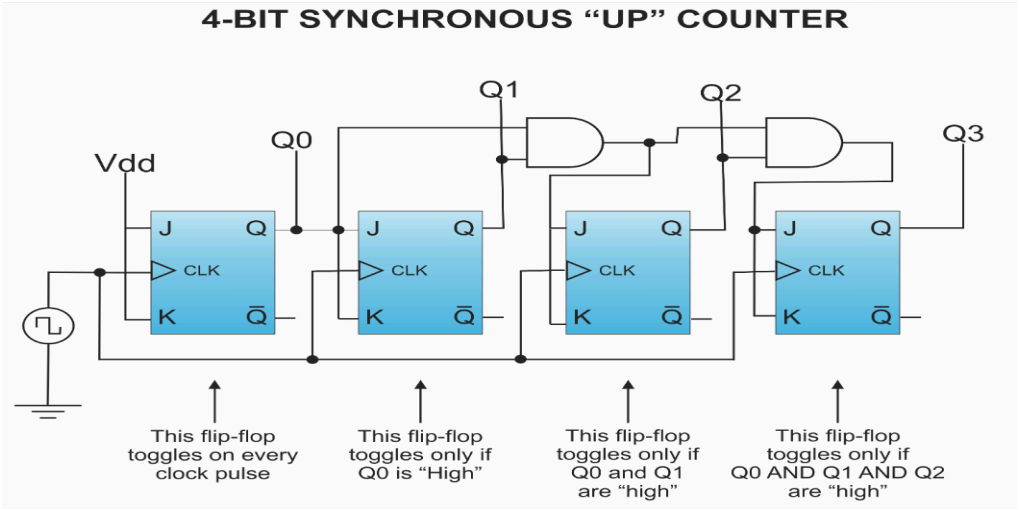
```
module johnson_counter_tb;
    reg clk, rst;
    wire [3:0] q;
johnson_counter uut (.clk(clk), .rst(rst), .q(q));
always #5 clk = ~clk;
initial begin
        clk = 0; rst = 1; #10;
        rst = 0;
   #100     $finish;
    end
endmodule
```

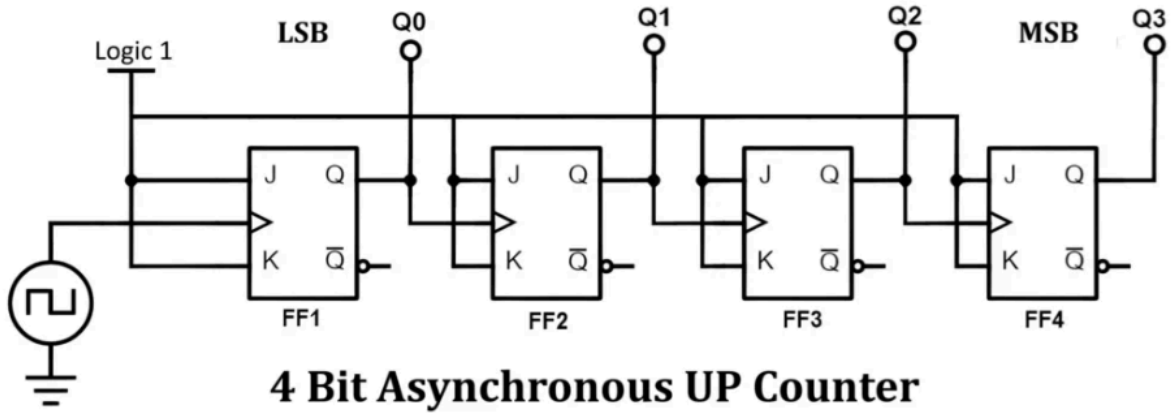
OUTPUT:



8) Synchronous and Asynchronous Counters

DESIGN:





VERILOG CODE:

```
// Synchronous 4-bit Up Counter
module sync_counter (
    input clk,
    input rst,
    output reg [3:0] q
```

```
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 4'b0;
        else q <= q + 1'b1;
    end
endmodule
// Asynchronous 4-bit Up Counter
module async_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 4'b0;
        else q[0] <= ~q[0];
    end
    always @(posedge q[0] or posedge rst) begin
        if (rst) q[1] <= 1'b0; else q[1] <= ~q[1];
    end
 always @(posedge q[1] or posedge rst) begin
        if (rst) q[2] <= 1'b0; else q[2] <= ~q[2];
    end
    always @(posedge q[2] or posedge rst) begin
        if (rst) q[3] <= 1'b0; else q[3] <= ~q[3];
    end
endmodule
```

TESTBENCH:

```
module counters_tb;
    reg clk, rst;
    wire [3:0] q_sync, q_async;

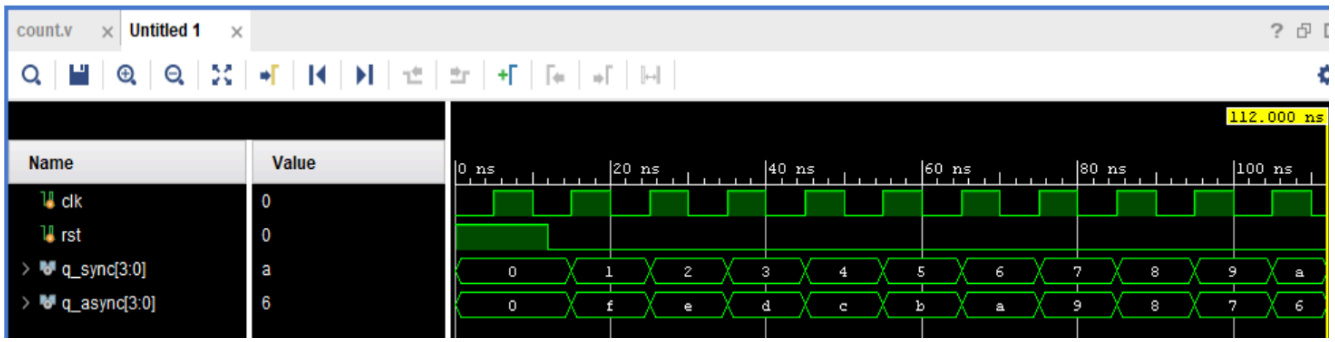
    sync_counter sync_inst (.clk(clk), .rst(rst), .q(q_sync));
    async_counter async_inst (.clk(clk), .rst(rst), .q(q_async));

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #12;
```

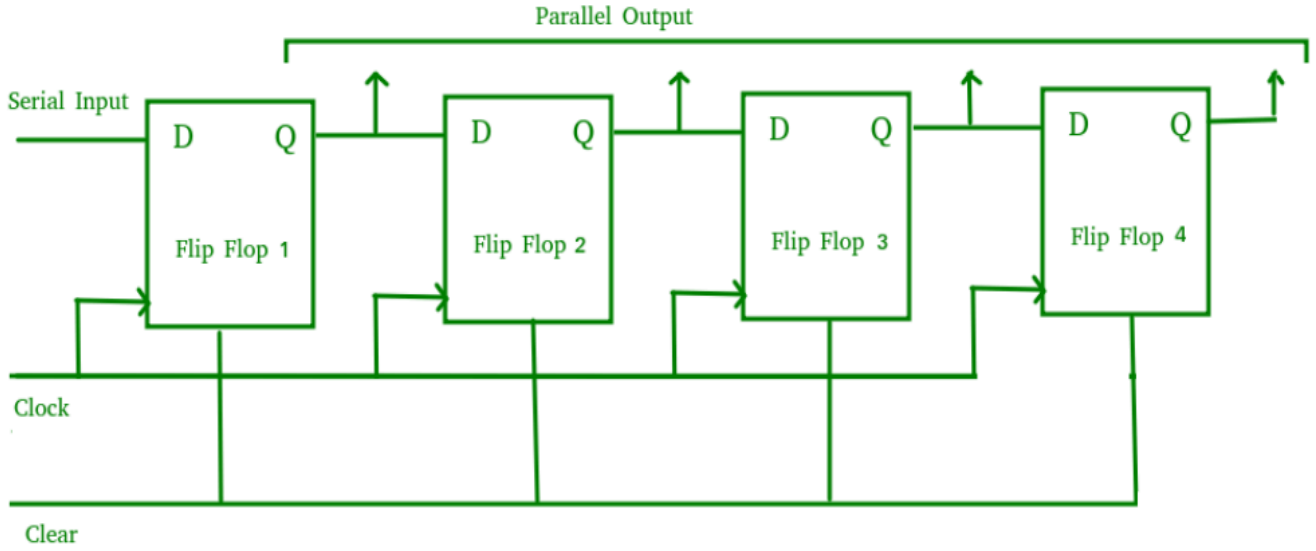
```
        rst = 0; #100;
        $finish;
    end
endmodule
```

OUTPUT:



9) Shift Register

DESIGN:



VERILOG CODE:

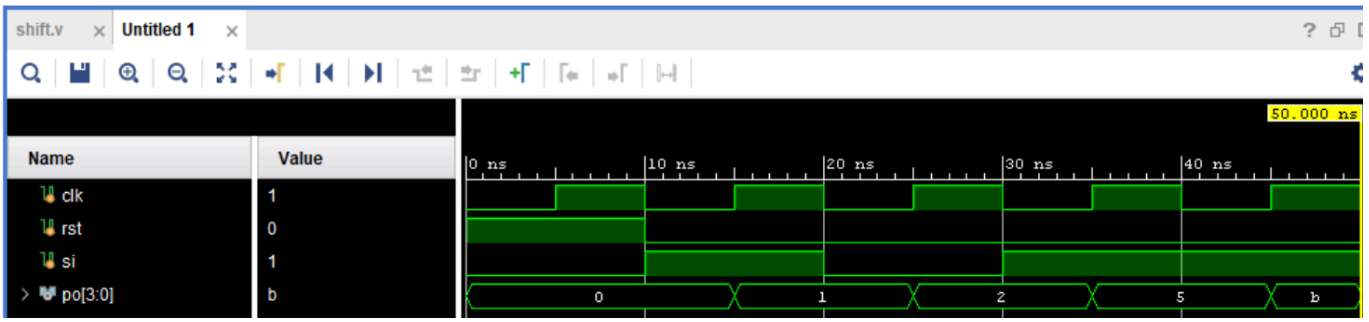
```
module shift_register (
    input clk,
    input rst,
    input si,
    output reg [3:0] po
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            po <= 4'b0000;
        else
            po <= {po[2:0], si};
```

```
    end
endmodule
```

TESTBENCH:

```
module shift_register_tb;
    reg clk, rst, si;
    wire [3:0] po;
    shift_register uut (.clk(clk), .rst(rst), .si(si), .po(po));
    always #5 clk = ~clk;
    initial begin
        clk = 0; rst = 1; si = 0; #10;
        rst = 0; si = 1; #10;
        si = 0; #10;
        si = 1; #10;
        si = 1; #10;
        $finish;
    end
endmodule
```

OUTPUT:



10) Divide-by-N Counter

VERILOG CODE:

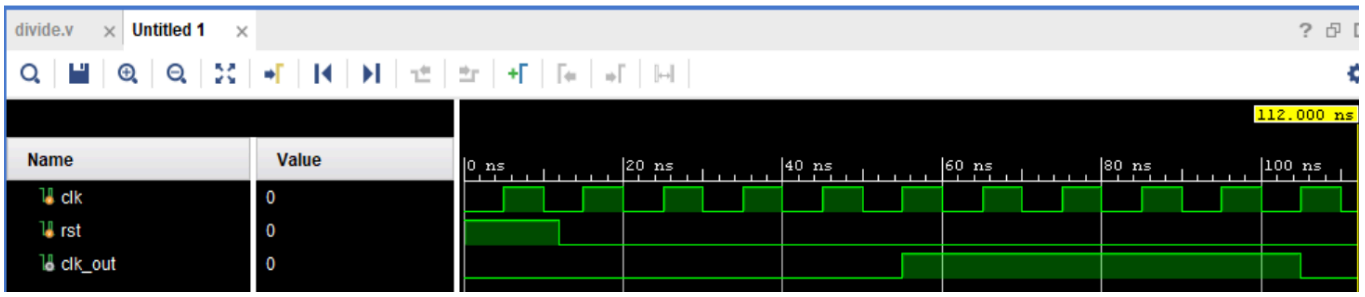
```
module divide_by_n #(parameter N = 5, parameter WIDTH = 3) (
    input clk,
    input rst,
    output reg clk_out
);
   reg [WIDTH-1:0] count;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            clk_out <= 0;
        end else if (count == N - 1) begin
```

```
            count <= 0;
            clk_out <= ~clk_out;
        end else begin
            count <= count + 1;
        end
    end
endmodule
```

TESTBENCH:

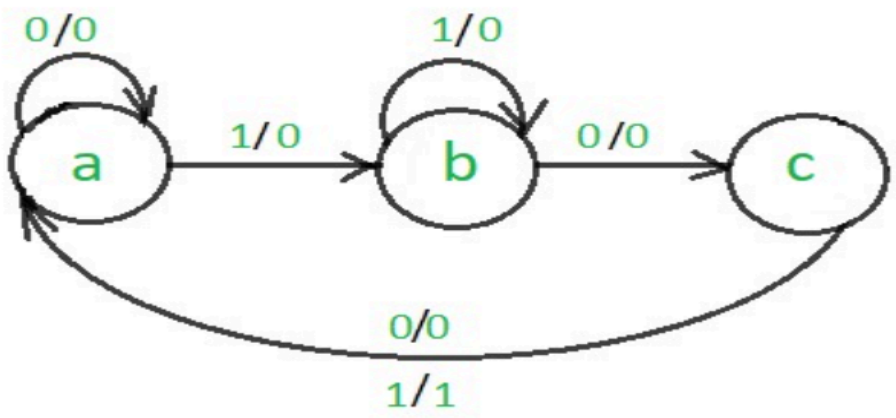
```
module divide_by_n_tb;
    reg clk, rst;
    wire clk_out;
 divide_by_n #(.N(5), .WIDTH(3)) uut (.clk(clk), .rst(rst), .clk_out(clk_out));
  always #5 clk = ~clk;
 initial begin
        clk = 0; rst = 1; #12;
        rst = 0; #100;
        $finish;
    end
endmodule
```

OUTPUT:

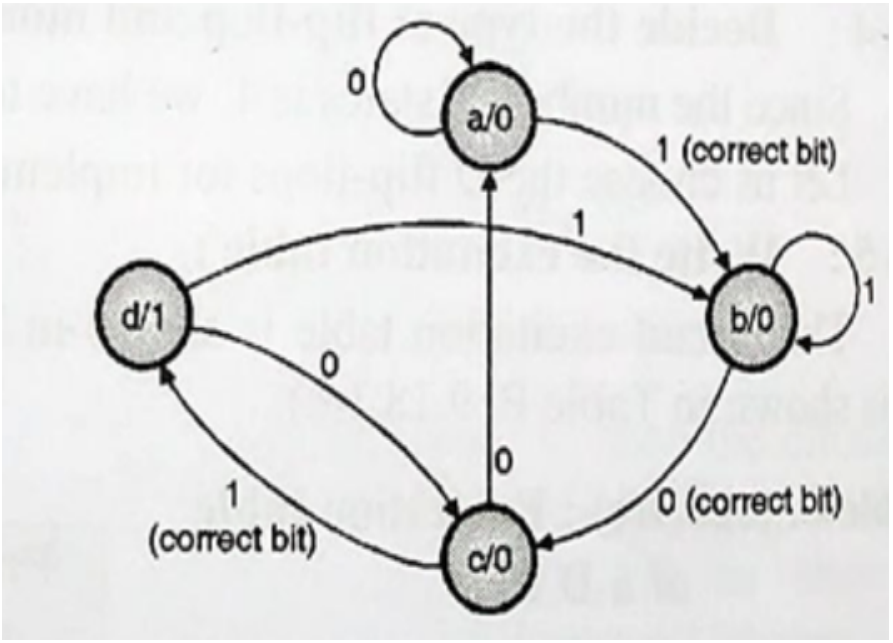


11) Mealy and Moore FSMs

DESIGN: Mealy machine(for 101)



DESIGN: Moore machine(for 101)



VERILOG CODE:

```
// Mealy Machine (Detects sequence "101")
module fsm_mealy (
    input clk,
    input rst,
    input in,
    output reg out
);
    parameter S0 = 1'b0, S1 = 1'b1;
    reg state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else state <= next_state;
    end

    always @(*) begin
        out = 0;
        case (state)
            S0: next_state = in ? S1 : S0;
        S1: begin
                if (in) next_state = S1;
                else begin next_state = S0; out = 1; end
            end
```

```
        endcase
    end
endmodule

// Moore Machine (Detects sequence "101")
module fsm_moore (
    input clk,
    input rst,
    input in,
    output reg out
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else state <= next_state;
    end
    always @(*) begin
        out = 0;
        case (state)
            S0: next_state = in ? S1 : S0;
            S1: next_state = in ? S1 : S2;
            S2: begin
                out = 1;
                next_state = in ? S1 : S0;
            end
        endcase
    end
endmodule
```

TESTBENCH:

```
module fsm_tb;
    reg clk, rst, in;
    wire out_mealy, out_moore;

    fsm_mealy m1 (.clk(clk), .rst(rst), .in(in), .out(out_mealy));
    fsm_moore m2 (.clk(clk), .rst(rst), .in(in), .out(out_moore));
```

```
    always #5 clk = ~clk;

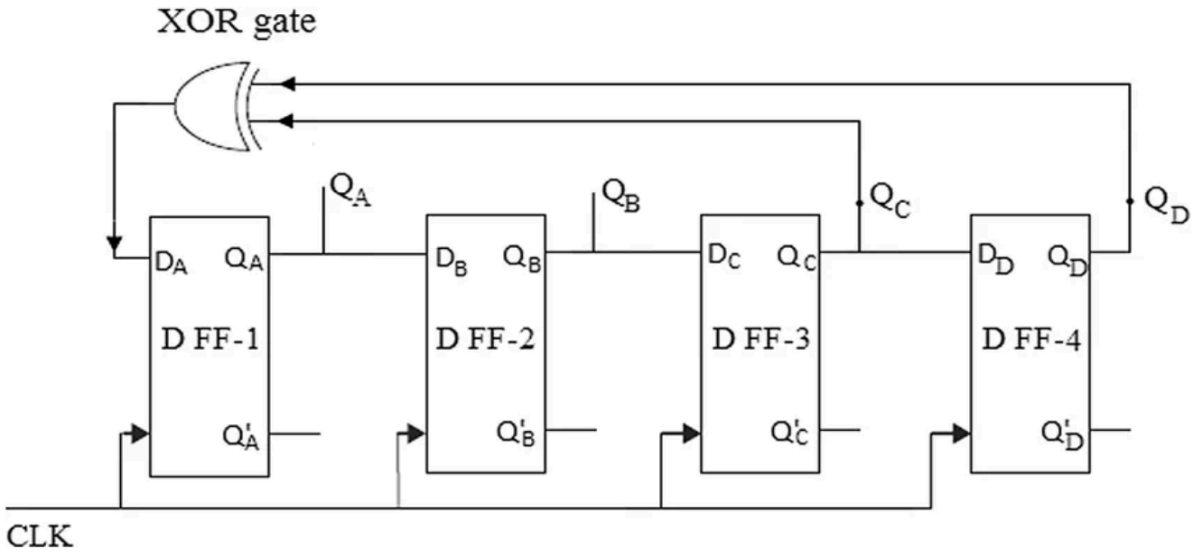
    initial begin
        clk = 0; rst = 1; in = 0; #12;
        rst = 0; in = 1; #10;
        in = 0; #10;
        in = 1; #10;
        in = 0; #10;
        $finish;
    end
endmodule
```

OUTPUT:



12) Random Sequence Generator (PRBS)

DESIGN:



VERILOG CODE:

```
module random_seq_generator (
    input clk,
    input rst,
    output reg [3:0] rand_out
);
    wire feedback;
    assign feedback = rand_out[3] ^ rand_out[2];
     always @(posedge clk or posedge rst) begin
        if (rst)
            rand_out <= 4'b1111; // Non-zero seed
        else
            rand_out <= {rand_out[2:0], feedback};
    end
endmodule
```

TESTBENCH:

```
module random_seq_generator_tb;
    reg clk, rst;
    wire [3:0] rand_out;

    random_seq_generator uut (.clk(clk), .rst(rst), .rand_out(rand_out));

    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #10;
        rst = 0; #150;
        $finish;
    end
endmodule
```

OUTPUT:

